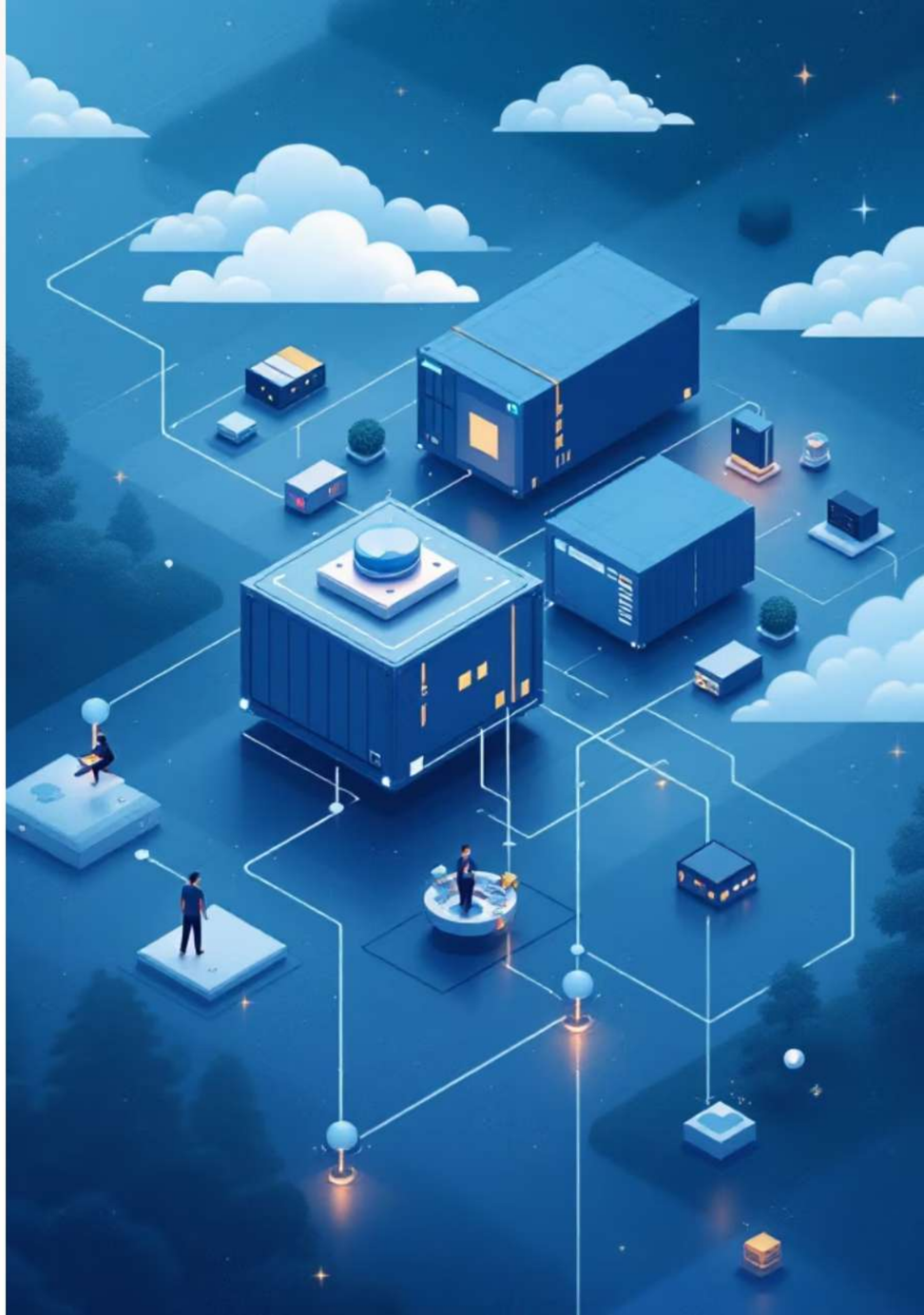
Parallel Test Execution Manager using Jenkins and Docker Selenium Grid

Student: Ankush Verma | **Course:** DevOps (CSE3253)

Reg No: 23FE10CSE00100



Why Automated Testing Matters

Modern software development demands rapid iteration and frequent releases. Automated testing ensures code quality while enabling teams to ship features faster without compromising reliability.

Traditional sequential testing becomes a bottleneck as test suites grow. **Parallel execution** is essential for maintaining fast feedback loops in CI/CD pipelines.

Problem Statement

Sequential Testing Bottleneck

Traditional Selenium tests execute one after another, creating long wait times that slow down development cycles and delay feedback.

Manual Infrastructure Management

Setting up and maintaining test environments manually is time-consuming and error-prone, especially when testing across multiple browsers and platforms.

Limited Scalability

As test suites expand with growing codebases, single-threaded execution becomes impractical, preventing teams from achieving fast iteration velocities.

Project Objectives

01

Automate Selenium Test Execution

Eliminate manual test triggering through automated pipeline integration

02

Implement Parallel Testing

Execute multiple test suites simultaneously using TestNG thread management

03

Containerize Selenium Grid

Create isolated, reproducible test environments with Docker containers

04

Integrate Jenkins CI/CD

Connect automated testing to complete DevOps pipeline workflow

05

Generate Test Reports

Produce detailed JUnit reports for analysis and tracking

Tools and Technologies

Docker

Containerization platform for creating Selenium Grid nodes and isolated test environments

Jenkins

CI/CD automation server orchestrating the entire testing pipeline workflow

Selenium Grid

Distributed testing framework enabling parallel execution across browser containers

Java & TestNG

Test framework providing annotations and parallel execution capabilities

GitHub

Version control system storing source code and test configurations

Apache Maven

Build automation tool managing dependencies and test execution lifecycle

System Architecture

GitHub Repository

Source code and test scripts stored in version control

Jenkins Pipeline

Automated workflow triggers and orchestrates testing process

Docker Selenium Grid

Dynamic container infrastructure for parallel test execution

Parallel Tests

TestNG executes multiple test suites simultaneously

JUnit Reports

Automated test result generation and analysis

Selenium Grid Architecture

Hub Node

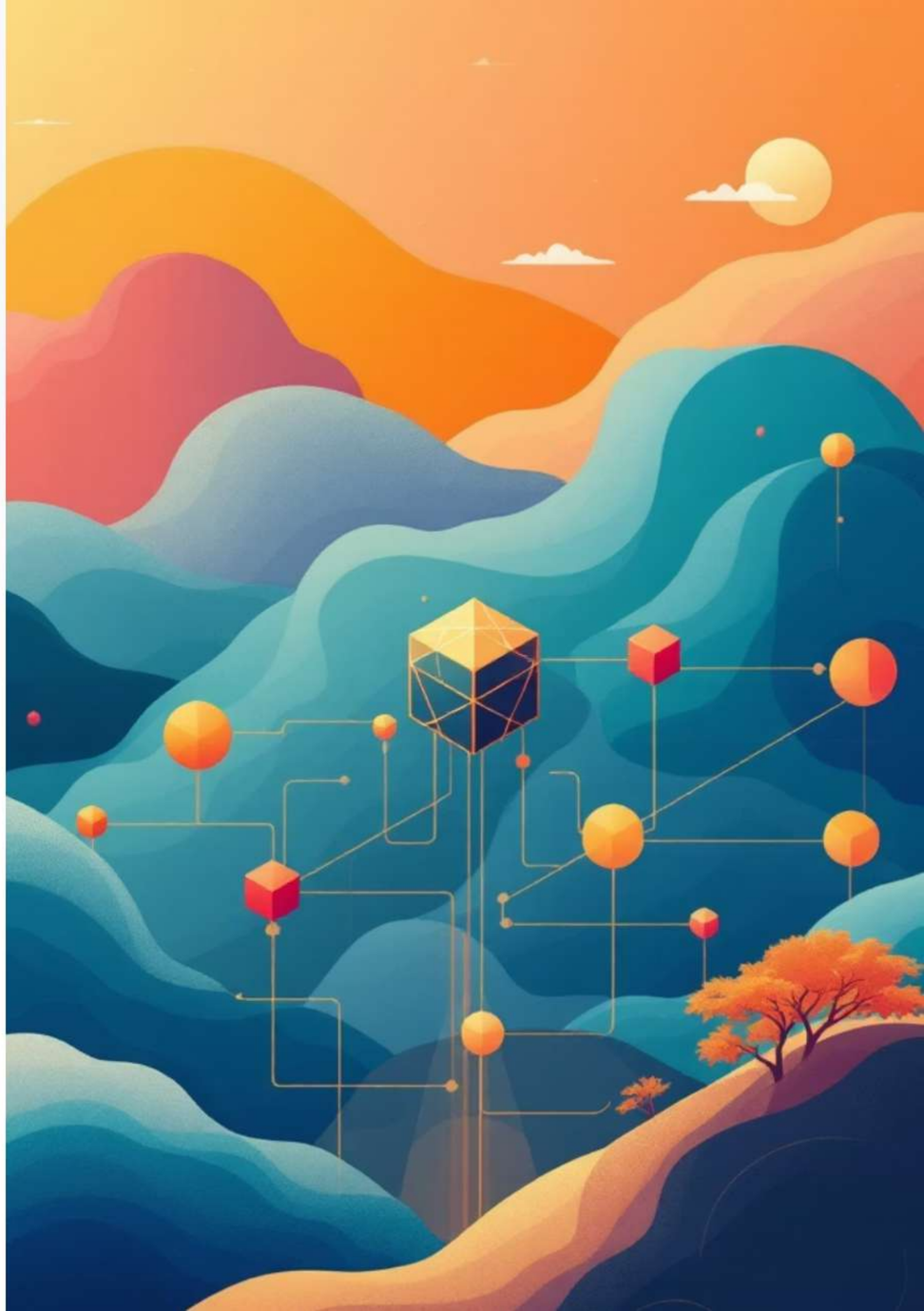
The Selenium Hub acts as the central coordinator, receiving test requests and distributing them to available browser nodes based on capabilities and availability.

It maintains a registry of available nodes and routes tests dynamically to optimize resource utilization.

Browser Nodes

Multiple Docker containers run Chrome browser instances, each registered with the Hub as available test execution environments.

Nodes execute tests independently and report results back to the Hub for aggregation and reporting.



Jenkins Pipeline Workflow

Stage 1: Start Grid

Execute `docker compose up` to launch Selenium Hub and Chrome node containers

Stage 2: Run Tests

Execute `mvn clean test` to trigger TestNG suite with parallel thread execution

Stage 3: Stop Grid

Execute `docker compose down` to gracefully shutdown all containers

Each pipeline stage includes error handling and logging to ensure reliability and provide detailed execution context.

Parallel Test Execution

TestNG Configuration

TestNG manages parallel execution through thread pools, distributing test methods across available threads based on configuration parameters.

Tests run simultaneously, each in its own thread, maximizing CPU utilization and reducing total execution time.

```xml
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd" >

<suite name="Parallel Suite" parallel="classes" thread-count="3">

    <test name="Parallel Tests">

        <classes>

            <class name="tests.LoginTest"/>
            <class name="tests.SearchTest"/>
            <class name="tests.CheckoutTest"/>

        </classes>

    </test>

</suite>
```

SearchTest

Validates search functionality in application

CheckoutTest

Tests checkout workflow and payment processing

Multiple test classes execute concurrently, each interacting with separate browser instances through the Selenium Grid.

Results and Conclusion

2	0	0
Tests Executed	**Failures**	**Errors**
SearchTest and CheckoutTest completed in parallel	All tests passed successfully	No execution errors encountered

---

Project Summary

This project demonstrates a complete DevOps solution for automated parallel test execution using Jenkins and Docker Selenium Grid. The system significantly reduces testing time compared to sequential execution while providing scalable, containerized infrastructure.

Key achievements: Automated CI/CD integration, parallel test execution with TestNG, dynamic Docker container management, and comprehensive test reporting through JUnit.